

Transaction System Processes (Cont.)

1 Lock Manager Evolution

In modern database systems, the role of the lock manager has evolved to reduce the overhead associated with interprocess communication.

Lock Table Access

To avoid the overhead of message-passing (RPCs) for every lock request, each **server process** operates directly on the **lock table** located in shared memory. This eliminates the need to send requests to a separate lock manager process for every transaction step.

Deadlock Detection

Even though processes update the lock table directly, the **Lock manager process** is still required. Its primary responsibility in this architecture is **deadlock detection**.

2 Mutual Exclusion Mechanisms

To ensure that no two processes access or modify the same shared data structure (like the lock table) simultaneously, database systems implement mutual exclusion using two primary levels of synchronization:

- **Atomic Hardware Instructions:** Lowest overhead; uses hardware support for immediate synchronization.
- **Operating System Semaphores:** Higher overhead due to context switching and OS involvement; used when a process needs to wait for a longer duration.

3 Atomic Hardware Instructions

Atomic instructions are used to implement **latches** (short-duration locks).

3.1 Test-and-Set (TAS)

How Test-and-Set Works

Test-and-Set uses a shared memory variable M , which represents a mutex. Initially, $M = 0$ (unlocked).

- **Atomic Operation:** Test-and-Set (M) atomically performs two actions:

1. Reads the current value of M
2. Sets $M = 1$

It then returns the **old value** of M .

- **Successful Lock Acquisition (Return = 0):** If the returned value is 0, the mutex was free and the process has successfully acquired the lock.
- **Failed Attempt (Return = 1):** If the returned value is 1, another process already holds the mutex. The process must keep retrying (busy waiting or *spinning*) until the lock is released.
- **Releasing the Lock:** The mutex is released by simply setting $M = 0$.

3.2 Compare-and-Swap (CAS)

How Compare-and-Swap Works

Compare-and-Swap is a more flexible atomic synchronization primitive.

If $M = V_1$, then set $M = V_2$ and return success; otherwise, return failure.

- **Atomicity:** The comparison and update occur as a single, indivisible operation.
- **Using CAS as a Mutex:** When $M = 0$ represents an unlocked state, $\text{CAS}(M, 0, 1)$ behaves exactly like Test-and-Set.
- **Ownership Tracking:** CAS allows storing more information than just locked/unlocked. For example, $\text{CAS}(M, 0, \text{ID})$ stores the **process or thread ID**, making it clear *who* currently owns the mutex.

4 Synchronization Workflows

Server processes follow strict protocols when interacting with the lock table in shared memory.

Lock Acquisition Steps

1. **Acquire Mutex:** Obtain a latch on the lock table using TAS or CAS.
2. **Check Allocation:**
 - If available: Update table to "allocated".
 - If conflict: Add request to the lock **queue**.
3. **Release Mutex:** Release the latch on the lock table.

Lock Release Steps

1. **Acquire Mutex:** Obtain a latch on the lock table.
2. **Remove Entry:** Remove the entry for the lock being released.
3. **Grant Pending:** If pending requests exist that can now be satisfied, mark them as "allocated".
4. **Release Mutex:** Release the latch on the lock table.